

Attention là Tất cả Những gì Bạn Cần

Ashish Vaswani*	Noam Shazeer*	Niki Parmar*	Jakob Uszkoreit*
Google Brain	Google Brain	Google Research	Google Research
avaswani@google.com	noam@google.com	nikip@google.com	usz@google.com
Llion Jones*	Aidan N. Gomez* [†]	Łukasz Kaiser*	
Google Research	University of Toronto	Google Brain	
llion@google.com	aidan@cs.toronto.edu	lukaszkaier@google.com	
Illia Polosukhin* [‡]			
illia.polosukhin@gmail.com			

Tóm tắt

Các mô hình chuyển đổi chuỗi (sequence transduction) ưu thế hiện nay dựa trên các mạng nơ-ron hồi tiếp (recurrent) hoặc tích chập (convolutional) phức tạp, bao gồm bộ mã hóa (encoder) và bộ giải mã (decoder). Các mô hình đạt hiệu quả tốt nhất cũng kết nối bộ mã hóa và bộ giải mã thông qua cơ chế chú ý (attention mechanism). Chúng tôi đề xuất một kiến trúc mạng mới đơn giản hơn, Transformer, dựa hoàn toàn trên cơ chế chú ý, loại bỏ hoàn toàn tính hồi tiếp và phép tích chập. Các thực nghiệm trên hai tác vụ dịch máy (machine translation) cho thấy các mô hình này vượt trội về chất lượng, đồng thời có khả năng song song hóa cao hơn và yêu cầu thời gian huấn luyện ít hơn đáng kể. Mô hình của chúng tôi đạt điểm BLEU là 28,4 trên tác vụ dịch Anh-Đức của WMT 2014, vượt qua kết quả tốt nhất hiện có, bao gồm cả các mô hình kết hợp (ensemble), hơn 2 điểm BLEU. Trên tác vụ dịch Anh-Pháp của WMT 2014, mô hình đơn lẻ của chúng tôi thiết lập điểm BLEU tiên tiến mới là 41,8 sau khi huấn luyện 3,5 ngày trên tám GPU, chỉ bằng một phần nhỏ chi phí huấn luyện của các mô hình tốt nhất trong tài liệu. Chúng tôi chứng minh rằng Transformer tổng quát hóa tốt cho các tác vụ khác bằng cách áp dụng thành công vào tác vụ phân tích cú pháp thành phần (constituency parsing) tiếng Anh, cả với dữ liệu huấn luyện lớn lẫn hạn chế.

*Đóng góp ngang nhau. Thứ tự liệt kê là ngẫu nhiên. Jakob đề xuất việc thay thế RNN bằng self-attention và khởi xướng nỗ lực đánh giá ý tưởng này. Ashish, cùng với Illia, đã thiết kế và triển khai các mô hình Transformer đầu tiên, và tham gia quan trọng vào mọi khía cạnh của công trình này. Noam đề xuất scaled dot-product attention, multi-head attention, và biểu diễn vị trí không có tham số, trở thành người tham gia vào hầu hết mọi chi tiết. Niki đã thiết kế, triển khai, tinh chỉnh và đánh giá vô số biến thể mô hình trong codebase gốc và tensor2tensor của chúng tôi. Llion cũng thử nghiệm các biến thể mô hình mới, chịu trách nhiệm về codebase ban đầu, suy luận hiệu quả và trực quan hóa. Lukasz và Aidan đã dành vô số ngày dài thiết kế các phần khác nhau và triển khai tensor2tensor, thay thế codebase trước đó của chúng tôi, cải thiện đáng kể kết quả và tăng tốc đáng kể nghiên cứu của chúng tôi.

[†] Công trình được thực hiện trong thời gian làm việc tại Google Brain.

[‡] Công trình được thực hiện trong thời gian làm việc tại Google Research.

Hội nghị lần thứ 31 về Hệ thống Xử lý Thông tin Thần kinh (NIPS 2017), Long Beach, CA, USA.
arXiv:1706.03762v7 [cs.CL] 2 Tháng 8 năm 2017

1 Giới thiệu

Mạng nơ-ron hồi tiếp (recurrent neural networks), đặc biệt là bộ nhớ ngắn hạn dài hạn (long short-term memory, LSTM) [13] và mạng nơ-ron hồi tiếp có cổng (gated recurrent neural networks) [7], đã được khẳng định vững chắc là các phương pháp tiên tiến nhất trong bài toán mô hình hóa chuỗi và chuyển đổi chuỗi, chẳng hạn như mô hình hóa ngôn ngữ và dịch máy [35, 2, 5]. Nhiều nỗ lực đã tiếp tục mở rộng ranh giới của các mô hình ngôn ngữ hồi tiếp và kiến trúc bộ mã hóa-bộ giải mã [38, 24, 15].

Các mô hình hồi tiếp thường thực hiện tính toán theo từng vị trí ký hiệu trong chuỗi đầu vào và đầu ra. Căn chỉnh các vị trí theo các bước trong thời gian tính toán, chúng tạo ra một chuỗi trạng thái ẩn h_t như một hàm của trạng thái ẩn trước đó h_{t-1} và đầu vào tại vị trí t . Bản chất tuần tự này về cơ bản ngăn cản việc song song hóa trong các mẫu huấn luyện, điều này trở nên nghiêm trọng ở các độ dài chuỗi lớn hơn, vì ràng buộc bộ nhớ hạn chế việc phân lô (batching) qua các mẫu. Công trình gần đây đã đạt được cải tiến đáng kể về hiệu quả tính toán thông qua các thủ thuật phân tích nhân tử (factorization tricks) [21] và tính toán có điều kiện (conditional computation) [32], đồng thời cải thiện hiệu suất mô hình trong trường hợp sau. Tuy nhiên, ràng buộc cơ bản của tính toán tuần tự vẫn còn tồn tại.

Cơ chế chú ý đã trở thành một phần không thể thiếu trong các mô hình mô hình hóa và chuyển đổi chuỗi hấp dẫn trong nhiều tác vụ, cho phép mô hình hóa các phụ thuộc mà không quan tâm đến khoảng cách của chúng trong chuỗi đầu vào hoặc đầu ra [2, 19]. Tuy nhiên, trong hầu hết các trường hợp trừ một vài ngoại lệ [27], cơ chế chú ý như vậy được sử dụng kết hợp với mạng hồi tiếp.

Trong công trình này, chúng tôi đề xuất Transformer, một kiến trúc mô hình loại bỏ tính hồi tiếp và thay vào đó dựa hoàn toàn vào cơ chế chú ý để nắm bắt các phụ thuộc toàn cục (global dependencies) giữa đầu vào và đầu ra. Transformer cho phép song song hóa nhiều hơn đáng kể và có thể đạt trình độ tiên tiến mới về chất lượng dịch thuật sau chỉ mười hai giờ huấn luyện trên tám GPU P100.

2 Nền tảng

Mục tiêu giảm thiểu tính toán tuần tự cũng là nền tảng của Extended Neural GPU [16], ByteNet [18] và ConvS2S [9], tất cả đều sử dụng mạng nơ-ron tích chập (convolutional neural networks) làm khối xây dựng cơ bản, tính toán các biểu diễn ẩn song song cho tất cả các vị trí đầu vào và đầu ra. Trong các mô hình này, số lượng phép toán cần thiết để liên hệ các tín hiệu từ hai vị trí đầu vào hoặc đầu ra tùy ý tăng theo khoảng cách giữa các vị trí, tuyến tính đối với ConvS2S và logarit đối với ByteNet. Điều này làm cho việc học các phụ thuộc giữa các vị trí xa nhau trở nên khó khăn hơn [12]. Trong Transformer, điều này được rút gọn xuống còn một số phép toán không đổi, mặc dù với cái giá là giảm độ phân giải hiệu quả do tính trung

bình các vị trí có trọng số chú ý, một hiệu ứng mà chúng tôi chống lại bằng Multi-Head Attention như mô tả trong phần 3.2.

Self-attention, đôi khi còn gọi là intra-attention, là cơ chế chú ý liên kết các vị trí khác nhau của một chuỗi đơn lẻ để tính toán biểu diễn của chuỗi đó. Self-attention đã được sử dụng thành công trong nhiều tác vụ khác nhau bao gồm đọc hiểu (reading comprehension), tóm tắt trừu tượng (abstractive summarization), kéo theo văn bản (textual entailment) và học biểu diễn câu độc lập với tác vụ [4, 27, 28, 22].

Mạng bộ nhớ đầu cuối (end-to-end memory networks) dựa trên cơ chế chú ý hồi tiếp thay vì tính hồi tiếp căn chỉnh theo chuỗi (sequence-aligned recurrence) và đã được chứng minh hoạt động tốt trên các tác vụ hỏi đáp ngôn ngữ đơn giản và mô hình hóa ngôn ngữ [34].

Theo hiểu biết tốt nhất của chúng tôi, Transformer là mô hình chuyển đổi đầu tiên dựa hoàn toàn vào self-attention để tính toán biểu diễn của đầu vào và đầu ra mà không sử dụng các RNN căn chỉnh theo chuỗi hoặc phép tích chập. Trong các phần tiếp theo, chúng tôi sẽ mô tả Transformer, đưa ra lý do cho self-attention và thảo luận về các ưu điểm của nó so với các mô hình như [17, 18] và [9].

3 Kiến trúc Mô hình

Hầu hết các mô hình chuyển đổi chuỗi nơ-ron cạnh tranh đều có cấu trúc bộ mã hóa-bộ giải mã (encoder-decoder) [5, 2, 35]. Ở đây, bộ mã hóa ánh xạ một chuỗi đầu vào gồm các biểu diễn ký hiệu $(x_1, \dots, x_n)$ thành một chuỗi các biểu diễn liên tục $\mathbf{z} = (z_1, \dots, z_n)$. Cho trước $\mathbf{z}$, bộ giải mã sau đó tạo ra một chuỗi đầu ra $(y_1, \dots, y_m)$ gồm các ký hiệu, mỗi lần một phần tử. Tại mỗi bước, mô hình là tự hồi quy (auto-regressive) [10], sử dụng các ký hiệu đã tạo trước đó làm đầu vào bổ sung khi tạo ký hiệu tiếp theo.

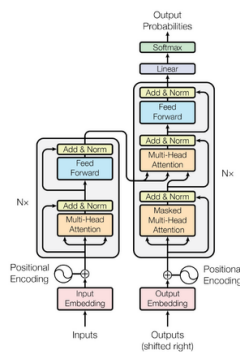


Figure 1: The Transformer - model architecture.

The Transformer follows this overall architecture using stacked self-attention and point-wise, fully connected layers for both the encoder and decoder, shown in the left and right halves of Figure 1 respectively.

3.1 Encoder and Decoder Stacks

Encoder: The encoder is composed of a stack of $N = 6$ identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. We employ a residual connection [11] around each of the two sub-layers, followed by layer normalization [1]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where x is the input to the sub-layer and $\text{Sublayer}(x)$ is the output of the sub-layer.

Hình 1. Transformer – kiến trúc mô hình.

Transformer tuân theo kiến trúc tổng thể này sử dụng các lớp self-attention xếp chồng và các lớp kết nối đầy đủ theo điểm (point-wise, fully connected layers) cho cả bộ mã hóa và bộ giải mã, được thể hiện ở nửa bên trái và bên phải của Hình 1, tương ứng.

3.1 Các Ngăn Xếp Bộ Mã hóa và Bộ Giải mã

Bộ mã hóa (Encoder): Bộ mã hóa được cấu thành từ một ngăn xếp gồm $N = 6$ lớp giống hệt nhau. Mỗi lớp có hai lớp con (sub-layers). Lớp con thứ nhất là cơ chế multi-head self-attention, và lớp con thứ hai là mạng truyền thẳng kết nối đầy đủ đơn giản theo vị trí (position-wise fully connected feed-forward network). Chúng tôi áp dụng kết nối phần dư (residual connection) [11] xung quanh mỗi trong hai lớp con, tiếp theo là chuẩn hóa lớp (layer normalization) [1]. Tức là, đầu ra của mỗi lớp con là $\text{LayerNorm}(x + \text{Sublayer}(x))$, trong đó $\text{Sublayer}(x)$ là hàm được thực hiện bởi chính lớp con đó. Để tạo điều kiện cho các kết nối phần dư này, tất cả các lớp con trong mô hình, cũng như các lớp nhúng (embedding layers), tạo ra đầu ra có chiều $d_{\text{model}} = 512$.

Bộ giải mã (Decoder): Bộ giải mã cũng được cấu thành từ một ngăn xếp gồm $N = 6$ lớp giống hệt nhau. Ngoài hai lớp con trong mỗi lớp bộ mã hóa, bộ giải mã chèn thêm một lớp con thứ ba, thực hiện multi-head attention trên đầu ra của ngăn xếp bộ mã hóa. Tương tự như bộ mã hóa, chúng tôi áp dụng các kết nối phần dư xung quanh mỗi lớp con, tiếp theo là chuẩn hóa lớp. Chúng tôi cũng điều chỉnh lớp con self-attention trong ngăn xếp bộ giải mã để ngăn các vị trí chú ý đến các vị trí tiếp theo. Việc che giấu (masking) này, kết hợp với việc các nhúng đầu ra được dịch chuyển một vị trí, đảm bảo rằng các dự đoán cho vị trí i chỉ có thể phụ thuộc vào các đầu ra đã biết tại các vị trí nhỏ hơn i .

3.2 Cơ chế Chú ý

Hàm chú ý (attention function) có thể được mô tả như một ánh xạ từ một truy vấn (query) và một tập hợp các cặp khóa-giá trị (key-value pairs) đến một đầu ra, trong đó truy vấn, khóa, giá trị và đầu ra đều là các vector. Đầu ra được tính như tổng trọng số của các giá trị, trong đó trọng số gán cho mỗi giá trị được tính bằng hàm tương thích (compatibility function) của truy vấn với khóa tương ứng.

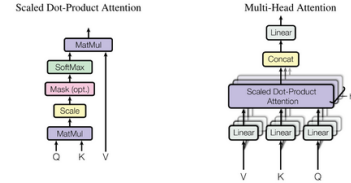


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

of the values, where the weight assigned to each value is computed by a compatibility function of the query with the corresponding key.

3.2.1 Scaled Dot-Product Attention

We call our particular attention “Scaled Dot-Product Attention” (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v . We compute the dot products of the query with all keys, divide each by $\sqrt{d_k}$, and apply a softmax function to obtain the weights on the values.

In practice, we compute the attention function on a set of queries simultaneously, stacked together

Hình 2. (trái) Scaled Dot-Product Attention. (phải) Multi-Head Attention bao gồm nhiều lớp chú ý chạy song song.

3.2.1 Scaled Dot-Product Attention (Chú ý Tích Điểm có Chuẩn hóa)

Chúng tôi gọi cơ chế chú ý cụ thể của mình là “Scaled Dot-Product Attention” (Hình 2). Đầu vào gồm các truy vấn và khóa có chiều d_k , và các giá trị có chiều d_v . Chúng tôi tính tích điểm của truy vấn với tất cả các khóa, chia mỗi giá trị cho $\sqrt{d_k}$, và áp dụng hàm softmax để thu được trọng số trên các giá trị.

Trong thực tế, chúng tôi tính hàm chú ý trên một tập hợp các truy vấn đồng thời, được gói vào một ma trận Q . Các khóa và giá trị cũng được gói vào các ma trận K và V . Chúng tôi tính ma trận đầu ra như sau:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Hai hàm chú ý được sử dụng phổ biến nhất là chú ý cộng (additive attention) [2] và chú ý tích điểm (dot-product/multiplicative attention). Chú ý tích điểm giống hệt thuật toán của chúng tôi, ngoại trừ thừa số tỷ lệ $\frac{1}{\sqrt{d_k}}$. Chú ý cộng tính hàm tương thích bằng mạng truyền thẳng với một lớp ẩn đơn. Mặc dù hai loại có độ phức tạp lý thuyết tương đương nhau, chú ý tích điểm nhanh hơn và hiệu quả bộ nhớ hơn trong thực tế, vì nó có thể được triển khai bằng mã nhân ma trận được tối ưu hóa cao.

Trong khi với các giá trị nhỏ của d_k thì hai cơ chế hoạt động tương tự nhau, chú ý cộng vượt trội hơn chú ý tích điểm không

có tỷ lệ (scaling) đối với các giá trị lớn hơn của d_k [3]. Chúng tôi nghi ngờ rằng đối với các giá trị lớn của d_k , các tích điểm tăng lớn về độ lớn, đẩy hàm softmax vào các vùng có gradient cực kỳ nhỏ.¹ Để chống lại hiệu ứng này, chúng tôi chia tỷ lệ các tích điểm bằng $\frac{1}{\sqrt{d_k}}$.

3.2.2 Multi-Head Attention (Chú ý Đa Đầu)

Thay vì thực hiện một hàm chú ý đơn lẻ với các khóa, giá trị và truy vấn d_{model} -chiều, chúng tôi nhận thấy có lợi khi chiếu tuyến tính các truy vấn, khóa và giá trị h lần với các phép chiếu tuyến tính học được khác nhau xuống các chiều d_k , d_k và d_v tương ứng. Trên mỗi phiên bản chiếu này của truy vấn, khóa và giá trị, chúng tôi thực hiện hàm chú ý song song, tạo ra các giá trị đầu ra d_v -chiều. Các giá trị này được nối (concatenated) và một lần nữa được chiếu, tạo ra các giá trị cuối cùng, như minh họa trong Hình 2.

Multi-head attention cho phép mô hình đồng thời chú ý đến thông tin từ các không gian con biểu diễn (representation subspaces) khác nhau tại các vị trí khác nhau. Với một đầu chú ý đơn lẻ, việc lấy trung bình sẽ cản trở điều này.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

trong đó $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$ (2)

Trong đó các phép chiếu là các ma trận tham số $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ và $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Trong công trình này, chúng tôi sử dụng $h = 8$ lớp chú ý song song hay đầu chú ý (heads). Với mỗi đầu, chúng tôi dùng $d_k = d_v = d_{\text{model}}/h = 64$. Do chiều của mỗi đầu bị giảm, tổng chi phí tính toán tương tự như chú ý đơn đầu với chiều đầy đủ.

3.2.3 Các Ứng dụng của Cơ chế Chú ý trong Mô hình

Transformer sử dụng multi-head attention theo ba cách khác nhau:

- Trong các lớp “encoder-decoder attention”, các truy vấn đến từ lớp bộ giải mã trước đó, và các khóa và giá trị bộ nhớ đến từ đầu ra của bộ mã hóa. Điều này cho phép mọi vị trí trong bộ giải mã chú ý đến tất cả các vị trí trong chuỗi đầu vào. Cơ chế này bắt chước các cơ chế encoder-decoder attention điển hình trong các mô hình sequence-to-sequence như [38, 2, 9].
- Bộ mã hóa chứa các lớp self-attention. Trong lớp self-attention, tất cả khóa, giá trị và truy vấn đến từ cùng một

nơi, trong trường hợp này là đầu ra của lớp trước trong bộ mã hóa. Mỗi vị trí trong bộ mã hóa có thể chú ý đến tất cả các vị trí trong lớp trước của bộ mã hóa.

- Tương tự, các lớp self-attention trong bộ giải mã cho phép mỗi vị trí trong bộ giải mã chú ý đến tất cả các vị trí trong bộ giải mã tính đến và bao gồm vị trí đó. Chúng tôi cần ngăn luồng thông tin hướng trái (leftward information flow) trong bộ giải mã để bảo toàn tính chất tự hồi quy. Chúng tôi triển khai điều này bên trong scaled dot-product attention bằng cách che giấu (đặt về $-\infty$) tất cả các giá trị trong đầu vào của softmax tương ứng với các kết nối không hợp lệ. Xem Hình 2.

3.3 Mạng Truyền thẳng Theo Vị trí

Ngoài các lớp con chú ý, mỗi lớp trong bộ mã hóa và bộ giải mã của chúng tôi chứa một mạng truyền thẳng kết nối đầy đủ (fully connected feed-forward network), được áp dụng cho mỗi vị trí một cách riêng biệt và giống hệt nhau. Mạng này gồm hai phép biến đổi tuyến tính với hàm kích hoạt ReLU ở giữa.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (3)$$

Trong khi các phép biến đổi tuyến tính giống nhau qua các vị trí khác nhau, chúng sử dụng các tham số khác nhau từ lớp này sang lớp khác. Một cách mô tả khác là xem đây là hai phép tích chập với kích thước nhân (kernel size) là 1. Chiều của đầu vào và đầu ra là $d_{\text{model}} = 512$, và lớp bên trong có chiều $d_{\text{ff}} = 2048$.

3.4 Nhúng từ và Softmax

Tương tự như các mô hình chuyển đổi chuỗi khác, chúng tôi sử dụng các nhúng (embeddings) học được để chuyển đổi các token đầu vào và đầu ra thành các vectơ có chiều d_{model} . Chúng tôi cũng sử dụng phép biến đổi tuyến tính học được thông thường và hàm softmax để chuyển đổi đầu ra của bộ giải mã thành các xác suất token tiếp theo được dự đoán. Trong mô hình của chúng tôi, chúng tôi chia sẻ cùng một ma trận trọng số giữa hai lớp nhúng và phép biến đổi tuyến tính trước softmax, tương tự như [30]. Trong các lớp nhúng, chúng tôi nhân các trọng số đó với $\sqrt{d_{\text{model}}}$.

3.5 Mã hóa Vị trí

Vì mô hình của chúng tôi không chứa phép hồi tiếp và phép tích chập, để mô hình có thể sử dụng thứ tự của chuỗi, chúng tôi phải đưa vào một số thông tin về vị trí tương đối hoặc tuyệt đối của các token trong chuỗi. Để đạt mục đích này, chúng tôi thêm “mã hóa vị trí” (positional encodings) vào các nhúng đầu vào tại đáy của các ngăn xếp bộ mã hóa và bộ giải mã. Các mã hóa vị trí có cùng chiều d_{model} như các nhúng, để hai thứ có thể được cộng vào nhau. Có nhiều lựa chọn mã hóa vị trí, học được và cố định [9].

¹Để minh họa tại sao các tích điểm trở nên lớn, giả sử rằng các thành phần của q và k là các biến ngẫu nhiên độc lập với trung bình 0 và phương sai 1. Khi đó tích điểm của chúng, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, có trung bình 0 và phương sai d_k .

Trong công trình này, chúng tôi sử dụng các hàm sin và cosine với các tần số khác nhau:

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{\text{model}}}) \quad (4)$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}}) \quad (5)$$

trong đó pos là vị trí và i là chiều. Tức là, mỗi chiều của mã hóa vị trí tương ứng với một đường sin. Các bước sóng (wavelengths) tạo thành một cấp số nhân hình học từ 2π đến $10000 \cdot 2\pi$. Chúng tôi chọn hàm này vì chúng tôi giả thuyết nó sẽ cho phép mô hình dễ dàng học cách chú ý theo vị trí tương đối, vì với mọi độ lệch cố định k , PE_{pos+k} có thể được biểu diễn như một hàm tuyến tính của PE_{pos} .

Chúng tôi cũng thử nghiệm sử dụng các nhúng vị trí học được [9] thay thế, và nhận thấy rằng hai phiên bản tạo ra kết quả gần như giống hệt nhau (xem Bảng 3 hàng (E)). Chúng tôi chọn phiên bản sin vì nó có thể cho phép mô hình ngoại suy (extrapolate) sang các độ dài chuỗi dài hơn những gì gặp phải trong quá trình huấn luyện.

4 Tại sao Self-Attention?

Trong phần này, chúng tôi so sánh các khía cạnh khác nhau của các lớp self-attention với các lớp hồi tiếp và tích chập thường được sử dụng để ánh xạ một chuỗi độ dài biến đổi gồm các biểu diễn ký hiệu $(x_1, \dots, x_n)$ sang một chuỗi khác có cùng độ dài $(z_1, \dots, z_n)$, với $x_i, z_i \in \mathbb{R}^d$, như một lớp ẩn trong bộ mã hóa hoặc bộ giải mã chuyển đổi chuỗi diễn hình. Lý giải cho việc sử dụng self-attention, chúng tôi xem xét ba tiêu chí mong muốn.

Một là tổng độ phức tạp tính toán (computational complexity) trên mỗi lớp. Hai là lượng tính toán có thể được song song hóa, được đo bằng số lượng phép toán tuần tự tối thiểu cần thiết.

Ba là độ dài đường đi giữa các phụ thuộc tầm xa trong mạng. Học các phụ thuộc tầm xa là thách thức chính trong nhiều tác vụ chuyển đổi chuỗi. Một yếu tố quan trọng ảnh hưởng đến khả năng học các phụ thuộc như vậy là độ dài của các đường đi mà các tín hiệu xuôi và ngược phải đi qua trong mạng. Các đường đi càng ngắn giữa bất kỳ tổ hợp vị trí nào trong chuỗi đầu vào và đầu ra, thì càng dễ học các phụ thuộc tầm xa [12]. Do đó, chúng tôi cũng so sánh độ dài đường đi tối đa giữa bất kỳ hai vị trí đầu vào và đầu ra nào trong các mạng gồm các loại lớp khác nhau.

Bảng 1. Độ dài đường đi tối đa, độ phức tạp trên mỗi lớp và số lượng phép toán tuần tự tối thiểu cho các loại lớp khác nhau. n là độ dài chuỗi, d là chiều biểu diễn, k là kích thước nhân của phép tích chập và r là kích thước vùng lân cận trong self-attention hạn chế.

Loại lớp mỗi lớp tuần tự đi tối đa	Độ phức tạp Phép toán Độ dài đường		
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Hồi tiếp	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Tích chập	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
[l]Self-Attention (hạn chế)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Như lưu ý trong Bảng 1, một lớp self-attention kết nối tất cả các vị trí với một số lượng phép toán thực hiện tuần tự không đổi, trong khi một lớp hồi tiếp yêu cầu $O(n)$ phép toán tuần tự. Về độ phức tạp tính toán, các lớp self-attention nhanh hơn các lớp hồi tiếp khi độ dài chuỗi n nhỏ hơn chiều biểu diễn d , đây là trường hợp thường gặp nhất với các biểu diễn câu được sử dụng bởi các mô hình tiên tiến nhất trong dịch máy, như biểu diễn word-piece [38] và byte-pair [31]. Để cải thiện hiệu suất tính toán cho các tác vụ liên quan đến các chuỗi rất dài, self-attention có thể bị hạn chế chỉ xem xét vùng lân cận có kích thước r trong chuỗi đầu vào xung quanh vị trí đầu ra tương ứng. Điều này sẽ tăng độ dài đường đi tối đa lên $O(n/r)$. Chúng tôi dự định điều tra phương pháp này thêm trong công trình tương lai.

Một lớp tích chập đơn lẻ với chiều rộng nhân $k < n$ không kết nối tất cả các cặp vị trí đầu vào và đầu ra. Làm như vậy đòi hỏi một ngăn xếp các lớp tích chập $O(n/k)$ trong trường hợp nhân liên tục (contiguous kernels), hoặc $O(\log_k(n))$ trong trường hợp tích chập giãn nở (dilated convolutions) [18], làm tăng độ dài của các đường đi dài nhất giữa bất kỳ hai vị trí nào trong mạng. Các lớp tích chập thường đắt hơn các lớp hồi tiếp, theo hệ số k . Tuy nhiên, các phép tích chập có thể tách (separable convolutions) [6] giảm đáng kể độ phức tạp xuống $O(k \cdot n \cdot d + n \cdot d^2)$. Tuy nhiên, ngay cả với $k = n$, độ phức tạp của một phép tích chập có thể tách bằng tổ hợp của một lớp self-attention và một lớp truyền thẳng theo điểm, đây là phương pháp chúng tôi áp dụng trong mô hình.

Một lợi ích phụ, self-attention có thể tạo ra các mô hình dễ diễn giải hơn. Chúng tôi kiểm tra các phân phối chú ý từ mô hình và trình bày, thảo luận các ví dụ trong phụ lục. Không chỉ các đầu chú ý (attention heads) riêng lẻ rõ ràng học cách thực hiện các tác vụ khác nhau, mà nhiều đầu còn thể hiện hành vi liên quan đến cấu trúc cú pháp và ngữ nghĩa của câu.

5 Huấn luyện

Phần này mô tả chế độ huấn luyện cho các mô hình của chúng tôi.

5.1 Dữ liệu Huấn luyện và Phân lô

Chúng tôi huấn luyện trên bộ dữ liệu chuẩn WMT 2014 Anh-Đức gồm khoảng 4,5 triệu cặp câu. Các câu được mã hóa bằng byte-pair encoding [3], có từ vựng nguồn-đích chung khoảng 37.000 token. Đối với tiếng Anh-Pháp, chúng tôi sử dụng bộ dữ liệu WMT 2014 Anh-Pháp lớn hơn đáng kể, gồm 36 triệu câu và chia token thành từ vựng word-piece 32.000 từ [38]. Các cặp câu được phân lô dựa trên độ dài chuỗi xấp xỉ. Mỗi lô huấn luyện chứa một tập hợp các cặp câu gồm khoảng 25.000 token nguồn và 25.000 token đích.

5.2 Phần cứng và Lịch trình

Chúng tôi huấn luyện các mô hình trên một máy với 8 GPU NVIDIA P100. Đối với các mô hình cơ sở (base models) của chúng tôi sử dụng các siêu tham số (hyperparameters) được mô tả xuyên suốt bài báo, mỗi bước huấn luyện mất khoảng 0,4 giây. Chúng tôi huấn luyện các mô hình cơ sở trong tổng cộng 100.000 bước hay 12 giờ. Đối với các mô hình lớn của chúng tôi (được mô tả ở dòng cuối cùng của Bảng 3), thời gian mỗi bước là 1,0 giây. Các mô hình lớn được huấn luyện trong 300.000 bước (3,5 ngày).

5.3 Bộ tối ưu hóa

Chúng tôi sử dụng bộ tối ưu hóa Adam [20] với $\beta_1 = 0,9$, $\beta_2 = 0,98$ và $\epsilon = 10^{-9}$. Chúng tôi thay đổi tốc độ học (learning rate) trong suốt quá trình huấn luyện, theo công thức:

$$\text{lrate} = d_{\text{model}}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5}) \quad (6)$$

Công thức này tương ứng với việc tăng tốc độ học tuyến tính trong warmup_steps bước huấn luyện đầu tiên, và sau đó giảm dần tỷ lệ nghịch với căn bậc hai của số bước. Chúng tôi sử dụng warmup_steps = 4000.

5.4 Điều chuẩn hóa

Chúng tôi áp dụng ba loại điều chuẩn hóa (regularization) trong quá trình huấn luyện:

Residual Dropout: Chúng tôi áp dụng dropout [33] vào đầu ra của mỗi lớp con, trước khi nó được cộng vào đầu vào lớp con và chuẩn hóa. Ngoài ra, chúng tôi áp dụng dropout vào tổng của các nhúng và mã hóa vị trí trong cả ngăn xếp bộ mã hóa và bộ giải mã. Đối với mô hình cơ sở, chúng tôi sử dụng tỷ lệ $P_{\text{drop}} = 0,1$.

Label Smoothing (Làm mịn nhãn): Trong quá trình huấn luyện, chúng tôi áp dụng label smoothing với giá trị $\epsilon_{\text{ls}} = 0,1$ [36]. Điều này làm tổn hại đến perplexity, vì mô hình học cách không chắc chắn hơn, nhưng cải thiện độ chính xác và điểm BLEU.

6 Kết quả

6.1 Dịch máy

Trên tác vụ dịch Anh-Đức của WMT 2014, mô hình transformer lớn (Transformer (big) trong Bảng 2) vượt qua các mô hình được báo cáo tốt nhất trước đó (bao gồm cả các mô hình kết hợp) hơn 2,0 điểm BLEU, thiết lập điểm BLEU tiên tiến mới là 28,4. Cấu hình của mô hình này được liệt kê ở dòng cuối cùng của Bảng 3. Quá trình huấn luyện mất 3,5 ngày trên 8 GPU P100. Ngay cả mô hình cơ sở của chúng tôi cũng vượt qua tất cả các mô hình và tổ hợp mô hình đã được công bố, với một phần nhỏ chi phí huấn luyện của bất kỳ mô hình cạnh tranh nào.

Trên tác vụ dịch Anh-Pháp của WMT 2014, mô hình lớn của chúng tôi đạt điểm BLEU là 41,0, vượt qua tất cả các mô hình đơn lẻ đã được công bố trước đó, với chi phí huấn luyện ít hơn 1/4 so với mô hình tiên tiến nhất trước đó. Mô hình Transformer (big) được huấn luyện cho tiếng Anh-Pháp sử dụng tỷ lệ dropout $P_{\text{drop}} = 0,1$, thay vì 0,3.

Đối với các mô hình cơ sở, chúng tôi sử dụng một mô hình đơn lẻ thu được bằng cách lấy trung bình 5 checkpoint cuối cùng, được ghi tại các khoảng thời gian 10 phút. Đối với các mô hình lớn, chúng tôi lấy trung bình 20 checkpoint cuối cùng. Chúng tôi sử dụng beam search với kích thước chùm (beam size) là 4 và phạt độ dài (length penalty) $\alpha = 0,6$ [38]. Các siêu tham số này được chọn sau khi thực nghiệm trên tập phát triển (development set). Chúng tôi đặt độ dài đầu ra tối đa trong quá trình suy luận bằng độ dài đầu vào + 50, nhưng kết thúc sớm khi có thể [38].

Bảng 2 tóm tắt kết quả của chúng tôi và so sánh chất lượng dịch thuật và chi phí huấn luyện với các kiến trúc mô hình khác trong tài liệu. Chúng tôi ước tính số phép toán dấu chấm động được sử dụng để huấn luyện một mô hình bằng cách nhân thời gian huấn luyện, số GPU được sử dụng và ước tính công suất điểm nổi đơn chính xác duy trì của mỗi GPU.²

6.2 Biến thể Mô hình

Để đánh giá tầm quan trọng của các thành phần khác nhau của Transformer, chúng tôi thay đổi mô hình cơ sở theo nhiều cách khác nhau, đo lường sự thay đổi hiệu suất dịch Anh-Đức trên tập phát triển newstest2013. Chúng tôi sử dụng beam search như mô tả trong phần trước, nhưng không lấy trung bình checkpoint. Chúng tôi trình bày các kết quả này trong Bảng 3.

Trong các hàng (A) của Bảng 3, chúng tôi thay đổi số lượng đầu chú ý và các chiều khóa và giá trị chú ý, giữ nguyên lượng tính toán không đổi, như mô tả trong phần 3.2.2. Trong khi chú ý đơn đầu tệ hơn 0,9 BLEU so với cài đặt tốt nhất, chất lượng cũng giảm với quá nhiều đầu.

Trong các hàng (B) của Bảng 3, chúng tôi quan sát thấy rằng

²Chúng tôi sử dụng các giá trị 2,8; 3,7; 6,0 và 9,5 TFLOPS cho K80, K40, M40 và P100, tương ứng.

Bảng 2. Transformer đạt điểm BLEU tốt hơn các mô hình tiên tiến nhất trước đó trên các bài kiểm tra newstest2014 Anh-Đức và Anh-Pháp, với một phần nhỏ chi phí huấn luyện.

Mô hình	BLEU		Chi phí huấn luyện (FLOPs)	
	AN-Đức	AN-Pháp	AN-Đức	AN-Pháp
ByteNet [18]	23,75			
Deep-Att + PosUnk [39]		39,2		$1,0 \cdot 10^{20}$
GNMT + RL [38]	24,6	39,92	$2,3 \cdot 10^{19}$	$1,4 \cdot 10^{20}$
ConvS2S [9]	25,16	40,46	$9,6 \cdot 10^{18}$	$1,5 \cdot 10^{20}$
MoE [32]	26,03	40,56	$2,0 \cdot 10^{19}$	$1,2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40,4		$8,0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26,30	41,16	$1,8 \cdot 10^{20}$	$1,1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26,36	41,29	$7,7 \cdot 10^{19}$	$1,2 \cdot 10^{21}$
Transformer (mô hình cơ sở)	27,3	38,1	$3,3 \cdot 10^{18}$	
Transformer (lớn)	28,4	41,8	$2,3 \cdot 10^{19}$	

giảm kích thước khóa chú ý d_k làm tổn hại đến chất lượng mô hình. Điều này cho thấy việc xác định tính tương thích không dễ dàng và một hàm tương thích phức tạp hơn tích điểm có thể có ích. Chúng tôi tiếp tục quan sát trong các hàng (C) và (D) rằng, như dự kiến, các mô hình lớn hơn tốt hơn và dropout rất hữu ích trong việc tránh quá khớp (overfitting). Trong hàng (E), chúng tôi thay thế mã hóa vị trí sin của chúng tôi bằng các nhúng vị trí học được [9], và quan sát kết quả gần như giống hệt với mô hình cơ sở.

6.3 Phân tích Cấu pháp Thành phần Tiếng Anh

Để đánh giá liệu Transformer có thể tổng quát hóa cho các tác vụ khác không, chúng tôi thực hiện các thực nghiệm về phân tích cú pháp thành phần (constituency parsing) tiếng Anh. Tác vụ này đặt ra các thách thức cụ thể: đầu ra phải tuân theo các ràng buộc cấu trúc mạnh mẽ và dài hơn đáng kể so với đầu vào. Hơn nữa, các mô hình sequence-to-sequence RNN chưa thể đạt được kết quả tiên tiến trong các chế độ dữ liệu nhỏ [37].

Chúng tôi đã huấn luyện một transformer 4 lớp với $d_{\text{model}} = 1024$ trên phần Wall Street Journal (WSJ) của Penn Treebank [25], khoảng 40.000 câu huấn luyện. Chúng tôi cũng huấn luyện trong cài đặt bán giám sát (semi-supervised), sử dụng các kho dữ liệu high-confidence và BerkeleyParser lớn hơn với khoảng 17 triệu câu [37]. Chúng tôi sử dụng từ vựng gồm 16.000 token cho cài đặt chỉ WSJ và từ vựng gồm 32.000 token cho cài đặt bán giám sát.

Chúng tôi chỉ thực hiện một số lượng nhỏ thực nghiệm để lựa chọn dropout, cả chú ý và phần dư (phần 5.4), tốc độ học và kích thước chum trên tập phát triển Section 22, tất cả các tham số khác vẫn không thay đổi so với mô hình dịch cơ sở Anh-Đức. Trong quá trình suy luận, chúng tôi tăng độ dài đầu ra tối đa lên độ dài đầu vào + 300. Chúng tôi sử dụng kích thước chum là 21 và $\alpha = 0,3$ cho cả cài đặt chỉ WSJ và bán giám sát.

Kết quả của chúng tôi trong Bảng 4 cho thấy rằng mặc dù thiếu tinh chỉnh theo tác vụ cụ thể, mô hình của chúng tôi hoạt động

tốt một cách đáng ngạc nhiên, mang lại kết quả tốt hơn tất cả các mô hình được báo cáo trước đó, ngoại trừ Recurrent Neural Network Grammar [8].

Trái với các mô hình sequence-to-sequence RNN [37], Transformer vượt qua BerkeleyParser [29] ngay cả khi chỉ huấn luyện trên tập huấn luyện WSJ gồm 40.000 câu.

Bảng 4. Transformer tổng quát hóa tốt cho tác vụ phân tích cú pháp thành phần tiếng Anh (Kết quả trên Section 23 của WSJ)

Bộ phân tích cú pháp	Dữ liệu huấn luyện	WSJ 23 F1
Vinyals & Kaiser et al. (2014) [37]	Chỉ WSJ, phân biệt	88,3
Petrov et al. (2006) [29]	Chỉ WSJ, phân biệt	90,4
Zhu et al. (2013) [40]	Chỉ WSJ, phân biệt	90,4
Dyer et al. (2016) [8]	Chỉ WSJ, phân biệt	91,7
Transformer (4 lớp)	Chỉ WSJ, phân biệt	91,3
Zhu et al. (2013) [40]	Bán giám sát	91,3
Huang & Harper (2009) [14]	Bán giám sát	91,3
McClosky et al. (2006) [26]	Bán giám sát	92,1
Vinyals & Kaiser et al. (2014) [37]	Bán giám sát	92,1
Transformer (4 lớp)	Bán giám sát	92,7
Luong et al. (2015) [23]	Đa tác vụ	93,0
Dyer et al. (2016) [8]	Sinh tạo	93,3

7 Kết luận

Trong công trình này, chúng tôi đã trình bày Transformer, mô hình chuyển đổi chuỗi đầu tiên dựa hoàn toàn vào cơ chế chú ý, thay thế các lớp hồi tiếp thường được sử dụng nhất trong các kiến trúc bộ mã hóa-bộ giải mã bằng multi-headed self-attention.

Đối với các tác vụ dịch thuật, Transformer có thể được huấn luyện nhanh hơn đáng kể so với các kiến trúc dựa trên các lớp hồi tiếp hoặc tích chập. Trên cả hai tác vụ dịch Anh-Đức và Anh-Pháp của WMT 2014, chúng tôi đạt trình độ tiên tiến

Bảng 3. Các biến thể kiến trúc Transformer. Các giá trị không liệt kê giống hệt với mô hình cơ sở. Tất cả số liệu đều trên tập phát triển dịch Anh-Đức, newstest2013. Perplexity được liệt kê là per-wordpiece, theo mã hóa byte-pair của chúng tôi, và không nên so sánh với perplexity per-word.

huấn luyện (dev) (dev)	N PPL BLEU	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	số bước		
cơ sở	6	512	2048	8	64	64	0,1	0,1	100K	4,92	25,8
(A)				1	512	512				5,29	24,9
				4	128	128				5,00	25,5
				16	32	32				4,91	25,8
				32	16	16				5,01	25,4
(B)					16					5,16	25,1
					32					5,01	25,4
(C)	2									6,11	23,7
	4									5,19	25,3
	8									4,88	25,5
		256			32	32				5,75	24,5
		1024			128	128				4,66	26,0
			1024							5,12	25,4
			4096							4,75	26,2
(D)							0,0			5,77	24,6
							0,2			4,95	25,5
								0,0		4,67	25,3
								0,2		5,47	25,7
(E)	những vị trí học được thay vì sin										25,7
lớn	6	1024	4096	16			0,3		300K	4,33	26,4

mới. Trong tác vụ trước, mô hình tốt nhất của chúng tôi vượt qua ngay cả tất cả các mô hình kết hợp được báo cáo trước đó. Chúng tôi rất hứng khởi với tương lai của các mô hình dựa trên chú ý và có kế hoạch áp dụng chúng cho các tác vụ khác. Chúng tôi có kế hoạch mở rộng Transformer sang các bài toán liên quan đến các phương thức đầu vào và đầu ra khác ngoài văn bản, và điều tra các cơ chế chú ý cục bộ, hạn chế để xử lý hiệu quả các đầu vào và đầu ra lớn như hình ảnh, âm thanh và video. Giảm thiểu tính tuần tự trong quá trình sinh tạo (generation) cũng là một mục tiêu nghiên cứu khác của chúng tôi.

Mã nguồn chúng tôi sử dụng để huấn luyện và đánh giá các mô hình có sẵn tại <https://github.com/tensorflow/tensor2tensor>.

Lời cảm ơn Chúng tôi biết ơn Nal Kalchbrenner và Stephan Gouws vì những nhận xét, chỉnh sửa và cảm hứng quý giá của họ.

Tài liệu

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *CoRR*, abs/1409.0473, 2014.
- [3] Denny Britz, Anna Goldie, Minh-Thang Luong, and Quoc V. Le. Massive exploration of neural machine translation architectures. *CoRR*, abs/1703.03906, 2017.
- [4] Jianpeng Cheng, Li Dong, and Mirella Lapata. Long short-term memory-networks for machine reading. *arXiv preprint arXiv:1601.06733*, 2016.
- [5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *CoRR*, abs/1406.1078, 2014.
- [6] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. *arXiv preprint arXiv:1610.02357*, 2016.
- [7] Junyoung Chung, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. Empirical evaluation of gated recurrent neural networks on sequence modeling. *CoRR*, abs/1412.3555, 2014.
- [8] Chris Dyer, Adhiguna Kuncoro, Miguel Ballesteros, and Noah A. Smith. Recurrent neural network grammars. In *Proc. of NAACL*, 2016.
- [9] Jonas Gehring, Michael Auli, David Grangier, Denis Yarats, and Yann N. Dauphin. Convolutional sequence to sequence learning. *arXiv preprint arXiv:1705.03122v2*, 2017.
- [10] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian

- Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [12] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, and Jürgen Schmidhuber. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.
- [13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [14] Zhongqiang Huang and Mary Harper. Self-training PCFG grammars with latent annotations across languages. In *Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing*, pages 832–841. ACL, August 2009.
- [15] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. Exploring the limits of language modeling. *arXiv preprint arXiv:1602.02410*, 2016.
- [16] Łukasz Kaiser and Samy Bengio. Can active memory replace attention? In *Advances in Neural Information Processing Systems*, (NIPS), 2016.
- [17] Łukasz Kaiser and Ilya Sutskever. Neural GPUs learn algorithms. In *International Conference on Learning Representations (ICLR)*, 2016.
- [18] Nal Kalchbrenner, Lasse Espeholt, Karen Simonyan, Aaron van den Oord, Alex Graves, and Koray Kavukcuoglu. Neural machine translation in linear time. *arXiv preprint arXiv:1610.10099v2*, 2017.
- [19] Yoon Kim, Carl Denton, Luong Hoang, and Alexander M. Rush. Structured attention networks. In *International Conference on Learning Representations*, 2017.
- [20] Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [21] Oleksii Kuchaiev and Boris Ginsburg. Factorization tricks for LSTM networks. *arXiv preprint arXiv:1703.10722*, 2017.
- [22] Zhouhan Lin, Minwei Feng, Cicero Nogueira dos Santos, Mo Yu, Bing Xiang, Bowen Zhou, and Yoshua Bengio. A structured self-attentive sentence embedding. *arXiv preprint arXiv:1703.03130*, 2017.
- [23] Minh-Thang Luong, Quoc V. Le, Ilya Sutskever, Oriol Vinyals, and Łukasz Kaiser. Multi-task sequence to sequence learning. *arXiv preprint arXiv:1511.06114*, 2015.
- [24] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. *arXiv preprint arXiv:1508.04025*, 2015.
- [25] Mitchell P Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of english: The penn treebank. *Computational linguistics*, 19(2):313–330, 1993.
- [26] David McClosky, Eugene Charniak, and Mark Johnson. Effective self-training for parsing. In *Proceedings of the Human Language Technology Conference of the NAACL, Main Conference*, pages 152–159. ACL, June 2006.
- [27] Ankur Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. A decomposable attention model. In *Empirical Methods in Natural Language Processing*, 2016.
- [28] Romain Paulus, Caiming Xiong, and Richard Socher. A deep reinforced model for abstractive summarization. *arXiv preprint arXiv:1705.04304*, 2017.
- [29] Slav Petrov, Leon Barrett, Romain Thibaux, and Dan Klein. Learning accurate, compact, and interpretable tree annotation. In *Proceedings of the 21st International Conference on Computational Linguistics and 44th Annual Meeting of the ACL*, pages 433–440. ACL, July 2006.
- [30] Ofir Press and Lior Wolf. Using the output embedding to improve language models. *arXiv preprint arXiv:1608.05859*, 2016.
- [31] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.
- [32] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- [33] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.
- [34] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 2440–2448. Curran Associates, Inc., 2015.
- [35] Ilya Sutskever, Oriol Vinyals, and Quoc VV Le. Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, pages 3104–3112, 2014.
- [36] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. *CoRR*, abs/1512.00567, 2015.
- [37] Vinyals & Kaiser, Koo, Petrov, Sutskever, and Hinton. Grammar as a foreign language. In *Advances in Neural*

Information Processing Systems, 2015.

- [38] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.
- [39] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. Deep recurrent models with fast-forward connections for neural machine translation. *CoRR*, abs/1606.04199, 2016.
- [40] Muhua Zhu, Yue Zhang, Wenliang Chen, Min Zhang, and Jingbo Zhu. Fast and accurate shift-reduce constituent parsing. In *Proceedings of the 51st Annual Meeting of the ACL (Volume 1: Long Papers)*, pages 434–443. ACL, August 2013.

Phụ lục: Trực quan hóa Cơ chế Chú ý

Attention Visualizations

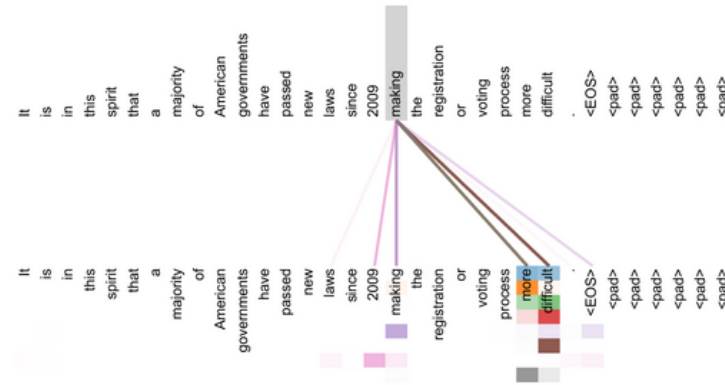


Figure 3: An example of the attention mechanism following long-distance dependencies in the encoder self-attention in layer 5 of 6. Many of the attention heads attend to a distant dependency of the verb 'making', completing the phrase 'making...more difficult'. Attentions here shown only for the word 'making'. Different colors represent different heads. Best viewed in color.

Hình 3. Ví dụ về cơ chế chú ý theo dõi các phụ thuộc tầm xa trong encoder self-attention tại lớp 5 trên 6. Nhiều đầu chú ý chú ý đến một phụ thuộc xa của động từ 'making', hoàn thành cụm từ 'making...more difficult'. Các chú ý chỉ được hiển thị cho từ 'making'. Các màu khác nhau đại diện cho các đầu khác nhau. Xem tốt nhất khi có màu sắc.

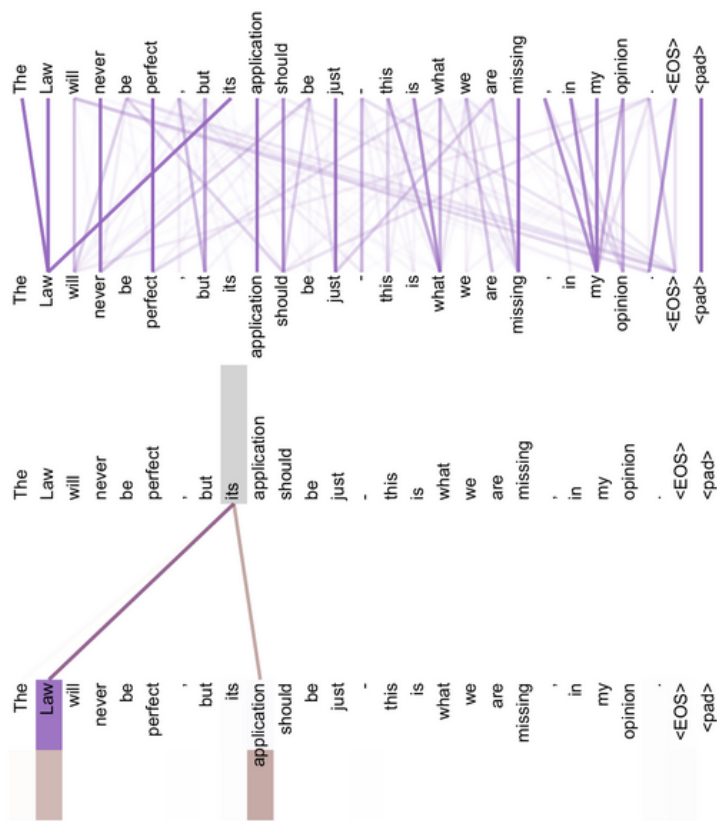


Figure 4: Two attention heads, also in layer 5 of 6, apparently involved in anaphora resolution. Top: Full attentions for head 5. Bottom: Isolated attentions from just the word 'its' for attention heads 5 and 6. Note that the attentions are very sharp for this word.

Hình 4. Hai đầu chú ý, cũng tại lớp 5 trên 6, rõ ràng tham gia vào giải quyết đại từ (anaphora resolution). Trên cùng: Toàn bộ chú ý cho đầu 5. Dưới cùng: Chú ý riêng lẻ chỉ từ từ 'its' cho các đầu chú ý 5 và 6. Lưu ý rằng các chú ý rất sắc nét cho từ này.

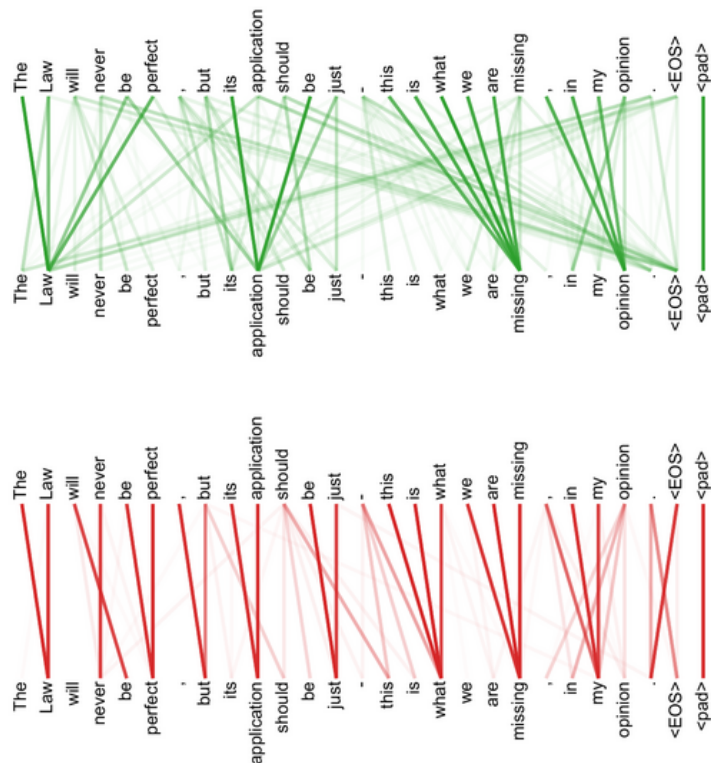


Figure 5: Many of the attention heads exhibit behaviour that seems related to the structure of the sentence. We give two such examples above, from two different heads from the encoder self-attention at layer 5 of 6. The heads clearly learned to perform different tasks.

Hình 5. Nhiều đầu chú ý thể hiện hành vi có vẻ liên quan đến cấu trúc của câu. Chúng tôi đưa ra hai ví dụ như vậy ở trên, từ hai đầu khác nhau của encoder self-attention tại lớp 5 trên 6. Các đầu rõ ràng học cách thực hiện các tác vụ khác nhau.